

Health Monitoring System using Embedded C and Data Structures

A PROJECT REPORT



Submitted by

Lakshman S	2022504515
Varun C M	2022504706
Dhivakar PCK	2022504036
Mohamed Raahil F	2022504505
Parthsarathi U	2022504547

BACHELOR OF ENGINEERING

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

MADRAS INSTITUTE OF TECHNOLOGY

ANNA UNIVERSITY

TABLE OF CONTENTS

Section No.	Title	Page No.
1	Introduction	4
2	Contribution	5
3	Architecture	5
4	Implementation Details	6
5	Data Structures Used	9
6	Results and Discussion	12
7	Achievements	14
8	Conclusion	16
9	References	16

Abstract

This project presents the design and implementation of a real-time healthcare monitoring system using embedded C and fundamental data structures. The system integrates multiple biomedical sensors—including a MAX30102 for measuring heart rate, SpO2 (oxygen saturation), and temperature, and a BMA400 accelerometer for motion detection—via the I2C protocol. An OLED display is used for real-time visualization of the sensor data, and Serial output provides extended monitoring capabilities.

To efficiently manage and analyze the continuously incoming sensor data, various data structures such as linked lists, queues, stacks, and binary search trees (BSTs) are utilized. The linked list serves as a log for historical data, the queue facilitates real-time streaming, the stack supports reverse-order access for recent readings, and the BST enables efficient searching and sorting based on timestamps.

The system also incorporates I2C bus recovery mechanisms to ensure robustness against communication failures. Designed for low-power microcontroller platforms, this project demonstrates how embedded C combined with classic data structures can provide a reliable and scalable solution for health monitoring in wearable or remote-care applications.

1. Introduction

With the growing demand for continuous health tracking and early detection of medical conditions, real-time healthcare monitoring systems have become increasingly vital, especially in remote and wearable applications. These systems enable users to monitor key physiological parameters such as heart rate, blood oxygen level (SpO2), body temperature, and physical activity, which are essential indicators of a person's health status.

This project focuses on the design and implementation of a real-time healthcare monitoring system using embedded C and standard data structures. The system integrates two primary sensors: the **MAX30102**, which measures heart rate, SpO2, and temperature, and the **BMA400**, a 3-axis accelerometer used to detect movement or posture changes. These sensors communicate with a microcontroller over the I2C protocol and output their readings to both a serial terminal and an OLED display for real-time feedback.

A unique feature of this system is the use of **data structures** to manage and process the collected sensor data efficiently. A **linked list** maintains a continuous log of all readings, while a **queue** supports real-time streaming by storing incoming data in a first-in-first-out (FIFO) manner. A **stack** enables last-in-first-out (LIFO) access for quickly reviewing the most recent data, and a **binary search tree (BST)** allows for efficient sorting and searching of data based on timestamps.

By combining sensor integration with efficient data management through data structures, this project not only provides accurate and timely health information but also demonstrates the power of algorithmic thinking in embedded systems. The system is designed with robustness in mind, featuring an I2C recovery mechanism to handle communication failures, making it reliable for continuous operation in practical healthcare environments.

Contributions

Lakshman S

Parthsarathi U

Varun C M

Dhivakar PCK

Mohamed Raahil F

All teammates equally contributed to the project by dividing the work. **Lakshman S** and **Mohamed Raahil F** focused on **software development**, while **Parthsarathi U**, **Varun C M**, and **Dhivakar PCK** handled **hardware implementation**. Tasks included **buying components, writing code, assembling the circuit, observing output, analyzing data, and documenting the process**.

2. Architecture

System Architecture

The healthcare monitoring system is designed around the **ESP32 microcontroller**, which serves as the central hub for sensor data acquisition, processing, and display. The system follows a modular architecture consisting of the following major layers:

2.1. Sensor Layer

- Collects physiological and motion-related data from biomedical sensors.
 - **MAX30102**: Measures heart rate, SpO2, and temperature.
 - **BMA400**: Detects motion and orientation through acceleration.

2.2. Processing Layer (ESP32)

- Manages communication with sensors via **I2C protocol**.
- Collects sensor readings periodically.
- Applies data structures (Linked List, Queue, Stack, BST) to organize and analyze data.
- Implements I2C bus recovery for fault tolerance.

2.3. Display & Output Layer

- Displays real-time data on a **128x64 OLED screen** (I2C).
- Sends extended data logs and debugging info to the **Serial Monitor** (USB/UART).

2.4. Storage & Analysis Layer

- Uses:
 - **Linked List** for logging all sensor entries.
 - **Queue** for streaming live data in FIFO order.
 - **Stack** to access recent readings (LIFO).
 - **Binary Search Tree (BST)** for sorted data access and timestamp-based search.
- May include future extensions for wireless data upload or SD card storage.

2.5 Components required

S.No	Component Name	Quantity	Description
1	ESP32 Dev Board	1	Microcontroller with Wi-Fi, Bluetooth, GPIO, I2C
2	MAX30102 Sensor Module	1	Measures heart rate, SpO2, and temperature
3	BMA400 Accelerometer	1	3-axis motion detection sensor
4	OLED Display (128x64, I2C)	1	For real-time data display
5	Breadboard	1	For circuit prototyping
6	Jumper Wires (Male-to-Male)	~10	For connections between components
7	USB Cable (Micro/Type-C)	1	For programming and Serial Monitor connection
8	Power Source (USB/Battery)	1	To power the ESP32 board

3. Implementation Details

3.1 Software Implementation

The software is developed in **C/C++** using the **Arduino IDE**, with the ESP32 board package installed. The implementation focuses on sensor data handling and organization using core **data structures**.

1. Sensor Initialization & Reading

- Libraries used:
 - Wire.h for I2C communication
 - DFRobot_BloodOxygen_S.h for MAX30102
 - SparkFun_BMA400_Arduino_Library.h for BMA400
 - Adafruit_SSD1306.h and Adafruit_GFX.h for OLED
- The ESP32 collects data:
 - Heart Rate and SpO2 via MAX30102.getHeartbeatSPO2()
 - Temperature via MAX30102.getTemperature_C()
 - Acceleration via accelerometer.getSensorData()

2. Data Processing Using Data Structures

- **Linked List:** Stores a complete log of sensor readings.
- **Queue:** Manages streaming data for real-time analysis.
- **Stack:** Allows access to the most recent data (LIFO).
- **Binary Search Tree (BST):** Stores data sorted by timestamps for efficient retrieval.

3. Data Display

- Real-time display on OLED using Adafruit_SSD1306.
- Formatted display includes heart rate, SpO2, temperature, and acceleration values.

4. I2C Bus Recovery

- Implements a recovery function to reset and reinitialize the I2C bus in case of communication failure with sensors.

5. Serial Output

- Outputs debug data and logs through Serial Monitor at 115200 baud rate.

6. Loop Function

- The loop() collects data every second.
- Data is stored into all four structures (log, stream, stack, BST).
- Every 10 seconds, sorted BST data is printed via Serial.

3.2 Hardware Implementation

The hardware implementation involves the physical setup of the system components connected to the **ESP32** microcontroller. Here's how the system is built:

1. Microcontroller Unit (ESP32)

- Acts as the central controller.
- Handles sensor communication, data processing, and display.

2. Sensor Connections

- **MAX30102 Sensor** (Heart Rate, SpO2, Temperature)
 - Connected via I2C:
 - SDA → GPIO 21
 - SCL → GPIO 22
 - VCC → 3.3V
 - GND → GND
- **BMA400 Accelerometer**
 - Also connected via I2C (same SDA and SCL lines).
 - Address conflict avoided by using one of two available I2C addresses.

3. Display Module

- **OLED Display (128x64, I2C)**
 - SDA → GPIO 21
 - SCL → GPIO 22
 - VCC → 3.3V
 - GND → GND

4. Power Supply

- Powered through USB cable from computer or portable battery for standalone use.

5. Prototyping Setup

- All components connected via a **breadboard** and **jumper wires**.
- Clean wire management and proper pull-up resistors for I2C lines if needed.

4. Data Structures Used and Their Implementation

To enhance the efficiency and performance of the health monitoring system, multiple data structures were used to optimize memory usage, enable real-time processing, and ensure systematic data management.

4.1 Structs (struct) – Organized Data Representation

Structs were used to group multiple sensor values into a single entity, ensuring clean and structured data handling. Each sensor's readings, timestamp, and status were encapsulated within a struct, making it easy to pass, retrieve, and process the data.

Usage in the Project:

- A struct stores heart rate, SpO₂, GSR, temperature, and accelerometer values along with a timestamp.
 - This struct is used for easy data management while processing sensor readings.
-

4.2 Linked List – Used for Sensor Data Logging

Purpose:

The linked list is used to maintain a complete log of all the sensor readings collected during the system's operation.

Why It Is Used:

In a dynamic environment where the number of readings is not fixed, a linked list is ideal. It does not require predefining the size of the dataset, allowing data to grow as needed. Each new sensor reading is added to the list, forming a chain of historical data points.

Advantages:

- **Dynamic Memory Allocation:** Allows the system to store as many readings as needed without worrying about memory waste or fixed array sizes.
- **Efficient Insertion:** New data can be quickly added at the beginning of the list.

- **Sequential Access:** Useful for iterating through the entire history of sensor data for review or export.

4.3 Queue (FIFO) – Used for Real-Time Data Streaming

Purpose:

The queue structure is used to manage sensor data in the order it arrives, particularly for streaming or sending data in real-time applications (e.g., remote monitoring or cloud upload).

Why It Is Used:

Queues operate on a First-In-First-Out basis, which is essential in real-time systems where data must be processed or transmitted in the order it was collected.

Advantages:

- **Preserves Order:** Ensures that data is processed in the correct sequence.
- **Buffering:** Acts as a temporary holding area for data that is waiting to be processed or transmitted.
- **Supports Streaming:** Essential for systems that need to stream data to a mobile app, server, or display.

4.4 Stack (LIFO) – Used for Accessing Recent Data

Purpose:

Stacks are used to retrieve the most recent data quickly, which can be helpful for immediate decision-making, like issuing alerts when critical thresholds are crossed.

Why It Is Used:

A stack follows Last-In-First-Out behavior, making it perfect for accessing the most recent data without scanning through the entire log.

Advantages:

- **Quick Access:** Provides immediate access to the latest reading.
- **Efficient:** Easy to implement with minimal overhead.
- **Ideal for Temporary Analysis:** Perfect for checking recent values against thresholds (e.g., high heart rate or low SpO2).

4.5 Binary Search Tree (BST) – Used for Time-Sorted Data Management

Purpose:

The BST is used to organize data based on timestamps, allowing for efficient storage and retrieval in a sorted manner.

Why It Is Used:

Unlike lists or stacks, BSTs allow data to be inserted and then accessed in sorted order. This is useful for applications where we need to analyze trends over time or retrieve specific time ranges.

Advantages:

- **Sorted Structure:** Automatically arranges data in ascending (or descending) order by timestamp.
- **Efficient Search:** Allows fast lookup, especially when searching by time range or for generating reports.
- **Good for Reporting:** Inorder traversal can be used to output data chronologically, which is helpful for user review or medical reporting.

4.6 Summary of Data Structures & Their Roles

Data Structure	Main Purpose	Why Used	Key Advantage
Linked List	Data logging	Dynamic storage of all readings	Flexible size, easy to traverse
Queue	Real-time data streaming	Maintains order of data for processing or transmission	FIFO structure suitable for streaming
Stack	Quick access to latest reading	Allows instant access to most recent value	LIFO structure ideal for recent data access
BST	Sorted data storage	Enables sorted access by timestamp	Efficient searching and chronological data retrieval

5.Results and Discussion

5.1 Observations from the Health Monitoring System

- **Stable Sensor Readings:**
The MAX30102 sensor provided consistent heart rate and SpO2 values when finger contact was proper. Body temperature readings from the same sensor remained within $\pm 0.5^{\circ}\text{C}$ of a reference thermometer.
- **Motion Detection:**
The BMA400 accelerometer detected changes in orientation and motion, which can be useful in identifying abnormal activity such as falls or tremors.
- **Real-Time Updates:**
Sensor data was processed every second and instantly displayed on the OLED screen, demonstrating real-time responsiveness.
- **Data Structure Effectiveness:**
 - The **linked list** helped retain all historical data in memory.
 - The **queue** preserved real-time streaming behavior.
 - The **stack** provided instant access to the latest values, useful for emergency thresholds.
 - The **BST** enabled sorted access, useful for time-based reports and analysis.

5.2 Performance Evaluation

Parameter	Observation
Sensor Accuracy	High, with minor variation depending on placement
Response Time	~1 second delay between reading and display
Memory Usage (ESP32 RAM)	Optimized through dynamic structures
System Stability	Stable after multiple hours of operation
Data Processing Speed	Efficient, even with increasing log size
Power Consumption	Moderate; ideal for short- to mid-term use

5.3 Comparison with Traditional Approaches

Compared to conventional methods where all sensor readings are logged sequentially, our approach using **DSA-based filtering and prioritization** proved to be more efficient:

Aspect	Traditional Systems	Proposed System (ESP32 + Data Structures)
Hardware	Bulky, hospital-grade devices	Compact, wearable, and low-cost hardware
Data Storage	Stored externally or not logged	In-memory logging using linked list and BST
Real-Time Processing	Limited or delayed	Real-time updates every second
Portability	Low	High (ESP32-based, suitable for wearables)
Flexibility	Hard-coded behavior	Modular, data-structure-driven logic
Memory Efficiency	Fixed size or file-based	Dynamic allocation using C structures
Accessibility of History	Limited	Supports on-device historical review (via BST/Stack)
Cost	High	Low and affordable for individual use

5.4 Limitations and Future

Scope Limitations:

While the healthcare monitoring system performs reliably in collecting and managing vital health data, certain limitations were observed during its development and testing:

- Sensor Sensitivity**
The accuracy of the heart rate and SpO2 readings depends heavily on proper finger placement and minimal motion. Small movements can lead to fluctuating or invalid readings.
- Limited Memory**
Since the ESP32 has limited onboard RAM, long-term data storage using dynamic structures (like linked lists or BSTs) may eventually exhaust memory, especially if the system runs continuously.
- No Persistent Storage**
Data is stored in memory only while the system is powered on. There is no external memory (e.g., SD card or flash storage) to retain data after a restart or power loss.

4. **Basic User Interface**

The OLED display provides essential data, but the user interface is minimal and lacks interaction or control features like menus, alerts, or historical graphing.

Future Scope:

This project can be significantly extended and enhanced in the future with both hardware and software improvements:

1. **Integration with Cloud Platforms**

Adding Wi-Fi or Bluetooth communication to sync data with cloud-based platforms or mobile applications will enable real-time remote patient monitoring.

2. **Persistent Data Storage**

Implementing SD card support or internal flash memory usage can allow long-term data logging and access to historical trends even after rebooting.

3. **Advanced Data Analytics**

Data structures like heaps or hash tables could be introduced for fast statistical analysis such as detecting anomalies, computing averages, or trend prediction.

6 Achievements

The development and implementation of the healthcare monitoring system using ESP32, embedded C, and data structures resulted in several notable achievements, both in terms of functionality and technical execution:

1. Successful Real-Time Monitoring of Vital Parameters

- Implemented accurate real-time acquisition of **heart rate**, **SpO2 (oxygen saturation)**, and **body temperature** using the MAX30102 sensor.
- Integrated **motion tracking** using the BMA400 accelerometer for activity and movement analysis.

2. Effective Use of Data Structures for Health Data Management

- Employed multiple data structures:
 - **Linked List** for complete sensor data logging.
 - **Queue** for real-time data streaming and processing.
 - **Stack** for instant access to the latest health readings.
 - **Binary Search Tree (BST)** for time-sorted data management and chronological analysis.

3. OLED Display Integration

- Successfully displayed live sensor readings on a compact OLED screen with a clear, user-friendly layout.

4. I2C Bus Recovery Mechanism

- Developed a custom I2C recovery function to handle communication failures between sensors and the ESP32, increasing system robustness.

5. Modular and Scalable Design

- Designed the system architecture in a modular way, enabling easy upgrades such as adding new sensors, storing data externally, or integrating cloud communication in the future.

6. Low-Cost, Portable Prototype

- Achieved a compact and cost-effective hardware setup suitable for wearable or personal health monitoring applications.

7 Conclusion

This project successfully demonstrates the design and implementation of a **real-time healthcare monitoring system** using the **ESP32 microcontroller**, essential biomedical sensors, and fundamental **data structures in C programming**. By integrating the **MAX30102** for heart rate, SpO2, and temperature, along with the **BMA400 accelerometer** for motion sensing, the system provides a compact and cost-effective solution for continuous health tracking.

The use of **data structures** such as **linked lists**, **queues**, **stacks**, and **binary search trees** played a vital role in efficient data collection, organization, and retrieval. These structures ensured smooth real-time operations, memory optimization, and scalability, showcasing how theoretical concepts from data structures can be applied practically in embedded systems.

The system achieved reliable sensor readings, effective data visualization through an OLED display, and robust I2C communication with recovery mechanisms. Though there are limitations such as the lack of persistent storage or wireless communication, the modular design offers significant **future scope** for enhancements like cloud connectivity, advanced analytics, and AI integration.

Overall, this project bridges embedded systems and software fundamentals to create a foundational platform for **portable, intelligent health monitoring**, and paves the way for further development in **IoT-based medical solutions**.

8 References

□ DFRobot MAX30102 Heart Rate and SpO2 Sensor Datasheet

https://wiki.dfrobot.com/SEN0344_MAX30102

□ Spark Fun BMA400 Accelerometer Hookup Guide

<https://learn.sparkfun.com/tutorials/sparkfun-triple-axis-accelerometer-bma400-hookup-guide>

□ Adafruit SSD1306 OLED Display Library Documentation

https://github.com/adafruit/Adafruit_SSD1306

□ ESP32 Technical Reference Manual

Espressif Systems – <https://www.espressif.com/en/products/socs/esp32/resources>